

BUG BOUNTY FIELD GUIDE

FOFA Dorking

FOFA Query Syntax, Real-World Dorks &
Automation for Bug Bounty Hunters and
Penetration Testers

- Fingerprint Search Engine
- 2025 Edition

Table of Contents

1. Introduction to FOFA

2. FOFA Query Syntax

2.1 Basic Syntax

2.2 Logical Operators

2.3 Special Operators

3. Core Search Fields

3.1 Domain & Host Fields

3.2 Content & Response Fields

3.3 Network & Meta Fields

3.4 Certificate & Fingerprint Fields

4. Bug Bounty Dorks by Category

4.1 Admin Panels & Consoles

4.2 Databases & Data Leaks

4.3 Cloud & DevOps Exposure

4.4 API Keys & Secrets

4.5 Vulnerable Services & CVEs

5. Targeting Specific Organizations

6. FOFA API & Automation

7. Integration with Bug Bounty Workflow

8. Best Practices & Legal

1. Introduction to FOFA

FOFA (Fingerprint Of All) is a Chinese-developed internet asset search engine that indexes billions of connected devices worldwide. It is particularly strong in Asia-Pacific regions where Shodan and Censys may have less coverage. For bug bounty hunters, FOFA is a powerful passive reconnaissance tool that discovers exposed assets, fingerprints technologies, and uncovers shadow IT. 🌐

FOFA excels at **fingerprinting** — identifying specific software versions, frameworks, and configurations through unique signatures in banners, HTML content, headers, and certificates. This makes it ideal for finding known vulnerable deployments at scale. 🔍

Why FOFA Matters for Bug Bounty

Global Coverage: Strong in APAC where other engines are weaker. **Rich Fingerprints:** FID (FOFA ID) and icon_hash for precise app identification. **Fuzzy Search:** FOFA supports fuzzy matching operators that other engines lack. **Speed:** Rapid indexing of newly exposed services. **Cost:** Competitive pricing for large-scale reconnaissance.

How FOFA Works

FOFA continuously scans IPv4 address space across multiple protocols, collecting banners, HTTP responses, certificate chains, and favicon data. Each asset is fingerprinted and stored in a searchable index. Users query this index using FOFA's structured query syntax to find specific technologies, versions, or misconfigurations.

FOFA Workflow

1. FOFA Scanners scan IPv4 + IPv6 space. **2.** Collect banners, certs, and HTML. **3.** Generate fingerprint IDs (FID). **4.** Index everything for search. **5.** Bug hunters query with CenQL syntax. **6.** Filter assets, validate, and report.

Account Tiers

Table 1 FOFA Account Tiers

Feature	Free	Registered	Advanced (F Points)
Search results	Limited	Expanded	Unlimited
API access	No	Basic	Full REST API

Fields supported	Basic	Most	All fields
Fuzzy search	Partial	Partial	Full fuzzy operators
Bulk export	No	Limited	CSV/JSON export

FOFA Access Notes

FOFA's main interface is at **fofa.info** (English) and **fofa.so** (Chinese). Free users see limited results. For serious bug bounty work, a registered account with F-Points is recommended. API access requires authentication with email/password or API key.

2. FOFA Query Syntax

FOFA uses a field-based query syntax similar to Shodan and Censys, but with unique operators like fuzzy search (`=`) and regex (`=~`). Mastering these operators is essential for effective dorking. 

2.1 Basic Syntax

SYNTAX

Table 2 Basic Syntax Rules

Operator	Meaning	Example
<code>field="value"</code>	Exact match	<code>domain="example.com"</code>
<code>field="value"</code>	Fuzzy match (contains)	<code>title="login"</code>
<code>field=~"regex"</code>	Regular expression	<code>ip=~"192\.\168\.\d+\.\d+"</code>
<code>field!="value"</code>	Not equal (exclude)	<code>port!="80"</code>
<code>field</code>	Field exists	<code>icon_hash</code>

Exact vs Fuzzy Match

`title="Login"` — exact match for the full title. `title="login"` — fuzzy match; title contains "login" anywhere. Use `=` for broader discovery and `=~` for precision.

2.2 Logical Operators

LOGIC

Table 3 Logical Operators

Operator	Meaning	Example
<code>&&</code>	AND — both conditions	<code>domain="example.com" && port="443"</code>
<code> </code>	OR — either condition	<code>title="Login" title="Admin"</code>
<code>()</code>	Grouping for precedence	<code>(port="80" port="443") && domain="example.com"</code>

2.3 Special Operators

ADVANCED

Table 4 Special Operators

Operator	Meaning	Example
<code>*</code>	Wildcard (fuzzy search only)	<code>host*="*.example.com"</code>
<code>?</code>	Single character wildcard	<code>host*="test?.example.com"</code>
<code>[]</code>	Character class in regex	<code>ip=~"10\[0-9]+\.[0-9]+\.[0-9]+"</code>
<code>" "</code>	Escape quotes inside values	<code>body="\password"</code>

Fuzzy Search Power

FOFA's fuzzy search with wildcards solves word-segmentation problems. For example, `banner="mysql version" && banner*="5.?.*"` finds MySQL versions matching 5.x.x pattern. This is unique to FOFA and incredibly useful for version enumeration.

3. Core Search Fields

3.1 Domain & Host Fields

DOMAIN RECON

Table 5 Domain & Host Fields

Field	Description	Example
domain	Base domain	domain="example.com"
host	Full hostname/subdomain	host="api.example.com"
host*	Fuzzy hostname match	host*="*.example.com"
ip	IPv4/IPv6 address	ip="203.0.113.42"
ip*	CIDR range (fuzzy)	ip*="203.0.113.0/24"
port	Port number	port="9200"
protocol	Protocol type	protocol="https"
is_domain	Has valid domain	is_domain="true"

3.2 Content & Response Fields

CONTENT

Table 6 Content & Response Fields

Field	Description	Example
title	HTML page title	title="Login"
body	HTTP response body	body="admin"
body*	Fuzzy body search	body*="password"
header	HTTP response headers	header="X-Powered-By"

banner	Service banner text	banner="0penSSH"
banner*	Fuzzy banner search	banner*="5.?.*"
fid	FOFA fingerprint ID	fid="sSxxxxxx"
icon_hash	Favicon hash (mmh3)	icon_hash="-335242539"

FID Explained

FID (FOFA ID) is a unique fingerprint identifier assigned to each web application or technology. Once FOFA identifies an app, it assigns an FID. Searching by FID finds **all instances** of that exact application version globally — perfect for finding shadow deployments. Example:

```
fid="sSxxxxxxxx"
```

3.3 Network & Meta Fields

NETWORK

Table 7 Network & Meta Fields

Field	Description	Example
country	Country code	country="US"
city	City name	city="Beijing"
region	Region/Province	region="California"
asn	AS number	asn="15169"
org	Organization name	org="Amazon"
server	Server header value	server="nginx"
os	Operating system	os="Windows"
app	Application name	app="Apache-Shiro"

3.4 Certificate & Fingerprint Fields

CERTIFICATE

Table 8 Certificate & Fingerprint Fields

Field	Description	Example
<code>cert</code>	Certificate content	<code>cert="example.com"</code>
<code>cert.is_valid</code>	Valid certificate	<code>cert.is_valid="true"</code>
<code>cert.issuer</code>	Certificate issuer	<code>cert.issuer="DigiCert"</code>
<code>cert.subject</code>	Certificate subject	<code>cert.subject="*.example.com"</code>
<code>jarm</code>	JARM fingerprint	<code>jarm="07d14d16d21d21d07c42d41d00041d24"</code>
<code>tls.version</code>	TLS version	<code>tls.version="TLSv1.2"</code>

JARM Fingerprints

JARM is an active TLS fingerprinting technique that identifies specific server software by its TLS handshake behavior. FOFA indexes JARM hashes, enabling identification of C2 servers, VPN gateways, and specific frameworks regardless of IP or domain. Example: Cobalt Strike has a known JARM hash `07d14d16d21d21d07c42d41d00041d24` .

4. Bug Bounty Dorks by Category

These FOFA queries are battle-tested for finding high-value bug bounty targets. Each dork includes the exact query, what it surfaces, and pro tips. 🕒

4.1 Admin Panels & Consoles

HIGH-IMPACT

ADMIN Jenkins Dashboard

Unprotected Jenkins instances expose builds, secrets, and often script console RCE.

```
title="Dashboard [Jenkins]" && body="jenkins"
```

ADMIN Apache Tomcat Manager

Tomcat manager interfaces allow WAR file deployment for remote code execution.

```
title="Apache Tomcat" && body="manager"
```

ADMIN phpMyAdmin

phpMyAdmin with weak credentials gives full database control and potential RCE.

```
title="phpMyAdmin" && body="pma_username"
```

ADMIN Grafana Login

Exposed Grafana dashboards reveal metrics, logs, and sometimes admin panels.

```
title="Grafana" && body="grafana"
```

ADMIN Kubernetes Dashboard

K8s dashboard exposure allows pod execution, log access, and cluster takeover.

```
title="Kubernetes Dashboard" && body="Kubernetes"
```

ADMIN Cisco ASDM / VPN

Cisco Adaptive Security Device Manager and VPN endpoints with known CVEs.

```
title="Cisco ASDM" && cert="cisco.com"
```

ADMIN Docker Registry

Unauthenticated Docker registries leak container images and source artifacts.

```
title="Docker Registry" && port="5000"
```

ADMIN Kibana Elasticsearch

Kibana interfaces expose indexed logs and allow Elasticsearch query execution.

```
title="Kibana" && port="5601"
```

4.2 Databases & Data Leaks

SENSITIVE DATA

DATABASE Open Elasticsearch

Unauthenticated Elasticsearch exposes all indexed data — critical P1 finding.

```
port="9200" && body="cluster_name" && body="indices"
```

DATABASE Exposed MongoDB

MongoDB without auth exposes entire databases. Port 27017 is the default.

```
port="27017" && body="MongoDB"
```

DATABASE Redis Unprotected

Redis without AUTH exposes cached data and may allow config rewrite for RCE.

```
port="6379" && banner="redis_version"
```

LEAKED **AWS S3 Buckets**

Misconfigured S3 buckets with public ListBucketResult XML. Check if intentional.

```
body="ListBucketResult" && body="s3.amazonaws.com"
```

LEAKED **Git Config Exposure**

Exposed .git/config files leak repo URLs, paths, and sometimes credentials.

```
body="[core]" && body="repositoryformatversion"
```

LEAKED **Open Directory Listings**

Directory indexes may expose backups, configs, databases, and source code files.

```
title="Index of /" && body="Parent Directory"
```

4.3 Cloud & DevOps Exposure

CLOUD INFRA

CLOUD **Swagger / OpenAPI Docs**

Exposed Swagger UI reveals API endpoints, schemas, and sometimes test tokens.

```
title="Swagger UI" && body="swagger-ui"
```

CLOUD **GraphQL Playground**

GraphQL introspection enabled allows full schema discovery and data extraction.

```
title="GraphQL Playground" && body="graphql"
```

CLOUD MinIO Storage Console

MinIO object storage interfaces may expose buckets without authentication.

```
title="MinIO Browser" && body="MinIO"
```

CLOUD Prometheus Metrics

Exposed Prometheus /metrics endpoints leak system metrics and sometimes paths.

```
body="process_resident_memory_bytes" || body="http_requests_total"
```

DEVOPS Docker-Compose / .env Leaks

Environment files in web roots leak API keys, DB credentials, and secrets.

```
body="DB_PASSWORD" && body="APP_KEY"
```

CLOUD Firebase / App Configs

Exposed mobile app configs and cloud function endpoints in HTTP responses.

```
body="firebaseio.com" || body="appspot.com"
```

4.4 API Keys & Secrets

SECRETS

SECRET Algolia API Keys

Algolia search API keys and application IDs exposed in frontend JS code.

```
body="algolia_api_key" || body="algolia_application_id"
```

SECRET **AWS Access Keys**

AWS access key IDs starting with AKIA or ASIA exposed in source code.

```
body="AKIA" && body="aws_access_key_id"
```

SECRET **Google API Keys**

Google Maps, Firebase, and Cloud API keys embedded in JavaScript.

```
body="AIza" && (body="googleapis" || body="firebase")
```

SECRET **Stripe Keys**

Stripe publishable and secret keys leaked in frontend or config files.

```
body="pk_live_" || body="sk_live_" || body="stripe.com"
```

SECRET **Slack / Discord Webhooks**

Webhook URLs exposed in source code allowing message injection.

```
body="hooks.slack.com" || body="discord.com/api/webhooks"
```

SECRET **JWT / Bearer Tokens**

Hardcoded JWT secrets or bearer tokens in frontend JS or configs.

```
body="Bearer " && body="eyJ"
```

4.5 Vulnerable Services & CVEs

CVE HUNTING

CVE Log4j / Log4Shell

Java applications with Log4j versions vulnerable to JNDI injection RCE.

```
body="log4j" && (body="jndi" || banner="log4j")
```

CVE Apache Struts (CVE-2017-5638)

Apache Struts with OGNL expression injection — still present on legacy systems.

```
app="Apache-Struts" && body="struts"
```

CVE F5 BIG-IP (CVE-2022-1388)

F5 BIG-IP iControl REST auth bypass — critical unauthenticated RCE.

```
title="BIG-IP" && app="BIG-IP" && port="443"
```

CVE GitLab RCE (CVE-2021-22205)

GitLab CE with ExifTool vulnerable to image upload RCE.

```
title="GitLab" && body="GitLab Community Edition"
```

CVE Spring4Shell (CVE-2022-22965)

Spring Framework on Tomcat with Java 9+ — WAR file write RCE.

```
body="Spring Boot" && body="springframework"
```

CVE PAN-OS GlobalProtect

Palo Alto Networks GlobalProtect with known XSS and command injection CVEs.

```
app="Palo-Alto-Networks" && title="GlobalProtect"
```

5. Targeting Specific Organizations

FOFA provides powerful ways to narrow reconnaissance to a specific organization. Combining domain, host, org, and certificate filters creates surgical precision. 🎯

Organization Enumeration Strategy

STEP-BY-STEP TARGETING

```
# Step 1: Base domain enumeration
domain="example.com"

# Step 2: Subdomain expansion via host wildcard
host*="*.example.com"

# Step 3: Find by certificate
cert="example.com"

# Step 4: Combine with organization name
org="Example Company" && (domain="example.com" || cert="example.com")

# Step 5: Exclude known CDNs and cloud providers
org="Example Company" && domain="example.com" && domain≠"amazonaws.com" &&
domain≠"cloudflare.com"

# Step 6: Find non-standard ports
host*="*.example.com" && port≠"80" && port≠"443" && port≠"8080"

# Step 7: HTTPS on non-443 ports (often dev/staging)
domain="example.com" && protocol="https" && port≠"443"
```

Favicon Hash Hunting

FOFA Icon Hash Technique

FOFA uses MurmurHash3 for favicon hashing. Click any favicon in FOFA results and the hash is automatically added to your query. This finds **all deployments** of the same application — including forgotten dev servers, shadow IT, and acquired properties.

FAVICON HASH WORKFLOW

```
# 1. Search target domain
domain="example.com"

# 2. Click favicon in results → FOFA auto-adds hash:
domain="example.com" && icon_hash="123456789"

# 3. Remove domain filter to find ALL instances globally:
icon_hash="123456789"

# 4. Combine with org for precision:
org="Example Company" && icon_hash="123456789"

# 5. Compute hash manually via Python:
python3 -c "
import sys, mmh3, base64
img = open('favicon.ico', 'rb').read()
print(mmh3.hash(base64.encodebytes(img)))
"
```

FID (FOFA ID) Hunting

FID Find All Instances by FID

FID uniquely identifies a specific app version. Search by FID to find every global deployment.

```
fid="sSxxxxxxx" && domain="example.com"
```

RECON Dev/Staging Discovery

Find non-standard ports and protocols that often host dev or staging environments.

```
domain="example.com" && protocol="https" && port≠"443" && port≠"80"
```

RECON HTTP on Non-80 Ports

HTTP services on unusual ports may serve admin panels, APIs, or legacy content.

```
domain="example.com" && protocol="http" && port≠"80" && port≠"443"
```


6. FOFA API & Automation

FOFA provides a REST API for programmatic access. For bug bounty hunters, automation is key — you need to scale reconnaissance across hundreds of targets. 🤖

API Authentication

API SETUP

```
# FOFA API endpoint
https://fofa.info/api/v1/search/all

# Required parameters:
# - email: your FOFA registered email
# - key: your API key (from account settings)
# - qbase64: Base64-encoded query string
# - size: number of results (max 10000 for paid, 100 for free)
# - page: pagination page number
# - fields: comma-separated output fields
```

Python API Script

PYTHON: FOFA TARGET ENUMERATION

```

import requests
import base64

EMAIL = "your@email.com"
API_KEY = "your_api_key"

# Base64 encode the query
query = 'domain="example.com" && title="Login"'
qbase64 = base64.b64encode(query.encode()).decode()

url = "https://fofa.info/api/v1/search/all"
params = {
    "email": EMAIL,
    "key": API_KEY,
    "qbase64": qbase64,
    "size": 100,
    "fields": "host,ip,port,title,header,server"
}

resp = requests.get(url, params=params, timeout=30)
data = resp.json()

if not data.get("error"):
    print(f"Total: {data['size']}")
    for item in data["results"]:
        host, ip, port, title, header, server = item
        print(f"{host} ({ip}:{port}) | {title}")
else:
    print(f"Error: {data}")

```

PYTHON: BULK DOMAIN RECON

```

import requests, base64, time

EMAIL = "your@email.com"
API_KEY = "your_api_key"

def fofa_search(query, size=100):
    qbase64 = base64.b64encode(query.encode()).decode()
    url = "https://fofa.info/api/v1/search/all"
    params = {
        "email": EMAIL,
        "key": API_KEY,
        "qbase64": qbase64,
        "size": size,
        "fields": "host,ip,port,title,domain"
    }
    resp = requests.get(url, params=params, timeout=30)
    return resp.json()

# Recon multiple targets
targets = ["target1.com", "target2.com", "target3.com"]
for target in targets:
    # Admin panels
    query = f'domain="{target}" && (title="admin" || title="login" || title="dashboard")'
    data = fofa_search(query, size=100)
    if not data.get("error") and int(data.get("size", 0)) > 0:
        print(f"\n[!] {target} - {data['size']} potential admin panels found")
        for item in data["results"][:5]:
            print(f"  → {item[0]} ({item[1]}:{item[2]}) | {item[3]}")
        time.sleep(1) # rate limiting

```

CLI One-Liners

CLI WORKFLOWS

```
# cURL example – query FOFA API
curl -s "https://fofa.info/api/v1/search/all?email=USER&key=KEY&qbase64=$(echo 'domain="example.com"' | base64)" | jq .

# Export hosts for httpx validation
python3 -c "
import requests, base64, sys
EMAIL='USER'; KEY='KEY'
q='domain=\"example.com\"'
q64=base64.b64encode(q.encode()).decode()
url=f'https://fofa.info/api/v1/search/all?email={EMAIL}&key={KEY}&qbase64={q64}&fields=host'
r=requests.get(url).json()
[print(x[0]) for x in r.get('results',[])]
" | tee fofa_hosts.txt | httpx -title -status-code

# Favicon hash query via API
python3 -c "
import requests, base64
EMAIL='USER'; KEY='KEY'
q='icon_hash=\"123456789\"'
q64=base64.b64encode(q.encode()).decode()
url=f'https://fofa.info/api/v1/search/all?email={EMAIL}&key={KEY}&qbase64={q64}&fields=host,ip,port,title'
r=requests.get(url).json()
print(f'Total: {r[\"size\"]}')
"
```

7. Integration with Bug Bounty Workflow

FOFA is most effective when chained with other tools. Here are proven workflows that professional hunters use daily. [🔗](#)

FOFA + Subfinder + dnsx + httpx Pipeline

FULL RECON CHAIN

```
# Phase 1: FOFA subdomain discovery via cert + host
python3 -c "
import requests, base64
EMAIL='USER'; KEY='KEY'
# Find all subdomains via host wildcard
q='host*=\"*.target.com\"'
q64=base64.b64encode(q.encode()).decode()
url=f'https://fofa.info/api/v1/search/all?email={EMAIL}&key={KEY}&qbase64={q64}&fields=host'
r=requests.get(url).json()
[print(x[0]) for x in r.get('results',[])]
" | sort -u | tee fofa_subs.txt

# Phase 2: Expand with Subfinder
subfinder -d target.com -all -silent | anew fofa_subs.txt

# Phase 3: Resolve to IPs
cat fofa_subs.txt | dnsx -a -silent | tee subs_resolved.txt

# Phase 4: Live validation with metadata
cat fofa_subs.txt | httpx -title -status-code -tech-detect -o live_http.txt

# Phase 5: FOFA deep scan on resolved IPs
for ip in $(cat subs_resolved.txt | awk '{print $2}' | sort -u | head -50); do
    python3 -c "
import requests, base64
EMAIL='USER'; KEY='KEY'
q=f'ip=\"{ip}\"'
q64=base64.b64encode(q.encode()).decode()
url=f'https://fofa.info/api/v1/search/all?email={EMAIL}&key={KEY}&qbase64={q64}&fields=port,title,banner'
r=requests.get(url).json()
print(f'=== {ip} ===')
[print(f'  {x[0]} | {x[1]} | {x[2][:50]}') for x in r.get('results',[])]
" >> fofa_ip_scan.txt
done
```

FOFA + Nuclei for CVE Validation

CVE HUNT CHAIN

```
# Step 1: FOFA query for specific product
python3 -c "
import requests, base64
EMAIL='USER'; KEY='KEY'
q='app=\"Jenkins\" && org=\"Target Inc\"'
q64=base64.b64encode(q.encode()).decode()
url=f'https://fofa.info/api/v1/search/all?email={EMAIL}&key={KEY}&qbase64={q64}&fields=host,ip,port'
r=requests.get(url).json()
[print(f'http://{x[1]}:{x[2]}' if x[2]≠'443' else f'https://{x[0]}') for x in r.get('results',[])]
" | tee jenkins_targets.txt

# Step 2: Validate with httprobe
cat jenkins_targets.txt | httprobe | tee live_jenkins.txt

# Step 3: Nuclei templates for Jenkins CVEs
cat live_jenkins.txt | nuclei -t cves/ -tags jenkins -severity critical,high
```

Continuous FOFA Monitoring

MONITOR SCRIPT

```
#!/usr/bin/env python3
# fofa_monitor.py – Daily FOFA asset monitor

import requests, base64, json, datetime, os

EMAIL = "your@email.com"
API_KEY = "your_api_key"
TARGET = "example.com"
STATE_FILE = "fofa_state.json"

def fofa_query(q, size=1000):
    q64 = base64.b64encode(q.encode()).decode()
    url = "https://fofa.info/api/v1/search/all"
    params = {
        "email": EMAIL, "key": API_KEY,
        "qbase64": q64, "size": size,
        "fields": "host,ip,port,title"
    }
    r = requests.get(url, params=params, timeout=30)
    return r.json()

# Load known state
try:
    with open(STATE_FILE) as f:
        known = set(json.load(f))
except:
    known = set()

# Query current state
query = f'domain="{TARGET}"'
data = fofa_query(query, size=1000)
current = set()
if not data.get("error"):
    for item in data.get("results", []):
        current.add(f"{item[1]}:{item[2]}")

# Find new assets
new = current - known
if new:
    print(f"[{datetime.datetime.now()}] NEW ASSETS for {TARGET}: {len(new)}")
    for asset in new:
        print(f"  - {asset}")
    # Send to notify, slack, or trigger scans

# Save state
with open(STATE_FILE, 'w') as f:
    json.dump(list(current), f)
```

Rate Limiting & Scope

FOFA API has rate limits. Free accounts: ~100 results/query. Paid accounts: up to 10,000 results/query. Always respect bug bounty program scope — FOFA is passive, but never probe discovered systems aggressively without permission.

8. Best Practices & Legal

Validation Checklist

Table 9 Before Reporting

Step	Action	Why
1	Confirm asset ownership via domain, cert, or org fields	CDNs and shared hosting may return false positives
2	Verify exposure is in program scope	Out-of-scope reports waste triager time
3	Check if already reported (HackerOne Hacktivity, etc.)	Duplicates don't earn bounties
4	Validate exploitability with manual testing	Confirmed findings get higher payouts
5	Document exact FOFA query used	Shows methodology and reproducibility
6	Include timestamp and screenshot from FOFA	Proves finding is current

Report Template

FOFA Finding Report Template

Title: [P1] Unauthenticated MongoDB Instance Belonging to [Target]

Description: During passive reconnaissance using FOFA, I discovered an exposed MongoDB instance associated with [Target Company].

FOFA Query: `port="27017" && banner="MongoDB" && domain="target.com"`

Asset: IP 203.0.113.42:27017 — confirmed live on [Date]

Impact: Unauthenticated MongoDB allows full CRUD operations on all databases. An attacker can read, modify, or delete sensitive data and potentially achieve RCE via MongoDB scripting.

Remediation: Enable authentication (bindIP + auth enabled). Restrict to internal network. Implement firewall rules blocking external port 27017.

Legal & Ethical Reminders

Do not exploit without authorization. FOFA is a passive search engine, but accessing discovered systems without permission violates CFAA, Computer Misuse Act, and similar laws.

Only report to programs where you have explicit written scope. Never modify data, upload files, or execute commands. Follow responsible disclosure timelines.

Recommended Resources

- **FOFA Search** — fofa.info / fofa.so (Official platforms)
- **FOFA API Docs** — en.fofa.info/api (REST API documentation)
- **FOFA Fuzzy Search Guide** — github.com/FofaInfo/FOFA-Library
- **ExploitDB** — exploit-db.com (CVE to exploit mapping)
- **Nuclei Templates** — github.com/projectdiscovery/nuclei-templates
- **KingOfBugBounty Tips** — github.com/KingOfBugbounty/KingOfBugBountyTips